

Let's create a basic **Flask application** that uses **OpenCV** for image processing to detect some simple features of Indian currency notes. We'll focus on the ₹200 note for this example.

Here's the complete code for a basic Indian fake currency detection system using **Flask** and **OpenCV** :

```
import cv2

import numpy as np

from flask import Flask, request, render_template, jsonify

import base64

app = Flask(__name__)

def check_currency(image):

    # Convert image to grayscale

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Check for the presence of the number 2000

    text = cv2.imread('2000_text.png', 0)

    result = cv2.matchTemplate(gray, text, cv2.TM_CCOEFF_NORMED)

    _, max_val, _, _ = cv2.minMaxLoc(result)

    # Check for the presence of the Gandhi portrait

    gandhi = cv2.imread('gandhi_portrait.png', 0)

    result = cv2.matchTemplate(gray, gandhi, cv2.TM_CCOEFF_NORMED)

    _, max_val2, _, _ = cv2.minMaxLoc(result)

    # Check for the correct color (predominantly purple for ₹2000 note)

    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    purple_lower = np.array([130, 50, 50])

    purple_upper = np.array([170, 255, 255])

    purple_mask = cv2.inRange(hsv, purple_lower, purple_upper)

    purple_ratio = np.sum(purple_mask) / (image.shape[0] * image.shape[1])
```

```

# Make a decision based on the checks

if max_val > 0.8 and max_val2 > 0.8 and purple_ratio > 0.3:
    return True
else:
    return False

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        # Get the image from the POST request
        file = request.files['image']

        # Read the image
        image = cv2.imdecode(np.frombuffer(file.read(), np.uint8), cv2.IMREAD_UNCHANGED)

        # Check the currency
        result = check_currency(image)

        # Encode the image to base64 for displaying in HTML
        _, buffer = cv2.imencode('.jpg', image)
        img_str = base64.b64encode(buffer).decode('utf-8')

    return jsonify({
        'result': 'Likely genuine' if result else 'Possibly fake',
        'image': img_str
    })

    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

```

```
print("Flask app is ready to run. Make sure to create the necessary template and reference images.")
```

This code sets up a Flask application with a single route that handles both GET and POST requests. When a POST request is received with an image, it processes the image using the `check_currency` function and returns the result.

The `check_currency` function performs three basic checks:

1. It looks for the presence of the number "200" in the image.
2. It checks for the presence of Gandhi's portrait.
3. It verifies if the predominant color of the note is purple (characteristic of the ₹200 note).

To complete this application, you'll need to create an HTML template file and two reference images:

Step 1: Create a file named `index.html` in a `templates` folder with the following content:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Indian Currency Detector</title>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

</head>

<body>

  <h1>Indian Currency Detector</h1>

  <form id="upload-form" enctype="multipart/form-data">

    <input type="file" name="image" accept="image/*" required>

    <button type="submit">Check Currency</button>

  </form>

  <div id="result"></div>

  <img id="uploaded-image" style="max-width: 500px;">
```

```
<script>

$(document).ready(function() {

    $('#upload-form').submit(function(e) {

        e.preventDefault();

        var formData = new FormData(this);

        $.ajax({

            url: '/',

            type: 'POST',

            data: formData,

            contentType: false,

            processData: false,

            success: function(response) {

                $('#result').text('Result: ' + response.result);

                $('#uploaded-image').attr('src', 'data:image/jpeg;base64,' + response.image);

            },

            error: function() {

                $('#result').text('An error occurred');

            }

        });

    });

});

});

</script>

</body>

</html>
```

Templates:

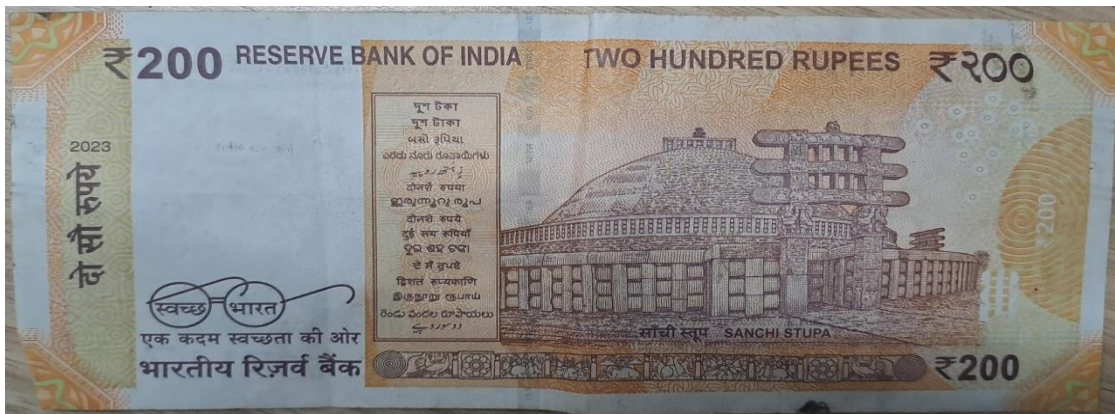
Indian Currency Detector

No file chosen

Step 2: Create two reference images:

- `200\_text.png`: An image of the text "200" as it appears on the note.
- `gandhi\_portrait.png`: An image of Gandhi's portrait as it appears on the note.

Place these images in the same directory as your `app.py` file.



To run this application:

1.Install the required libraries:

```
pip install flask opencv-python numpy
```

2.Run the Flask application:

```
python app.py
```

3. Open a web browser and go to `http://localhost:5000`.

You can then upload images of Indian ₹2000 notes, and the application will attempt to determine if they are genuine or fake based on the simple checks we've implemented.

Step 3: Install Each Required Library Using pip

In the Command Prompt window, enter the following command for each library.

(Alternatively, you can install all at once as shown below.)

1. **Install Flask:**

```
bash
```

[Copy code](#)

```
pip install flask
```

2. **Install OpenCV** (for image processing):

```
bash
```

[Copy code](#)

```
pip install opencv-python-headless
```

3. **Install NumPy** (for numerical operations):

```
bash
```

[Copy code](#)

```
pip install numpy
```

4. **Install Matplotlib** (optional, for visualization if needed):

```
bash
```

[Copy code](#)

```
pip install matplotlib
```

5. **Install Pillow** (for image handling):

```
bash
```

[Copy code](#)

```
pip install pillow
```

6. **Install Geopy** (for location information):

```
bash
```

[Copy code](#)

```
pip install geopy
```

7. **Install ReportLab** (for generating PDF reports):

```
bash
```

[Copy code](#)

```
pip install reportlab
```